Assignment 1: Bare-Metal Boot-Up of STM32F4 on QEMU

Course: Embedded Systems
Topic: Bare-Metal Boot Sequence and Runtime Initialization
Target MCU: STM32F4 (Cortex-M4F)
Execution Platform: QEMU (No Physical Development Board)

Marks – 20

Due date: 11:59pm, Sunday, 12th April, 2026

You may cheat only at your peril since any breach of academic integrity guidelines will attract the strictest possible action and penalty.

You may rest assured AI engines will not do this assignment. They produced crack induced garbage for me.

---

1. Objective

The objective of this assignment is to understand and implement the **bare-metal boot sequence** of an STM32F4 microcontroller, starting from **reset** and ending at the execution of `main()`.

You will implement the startup code, runtime initialization, linker script, and interrupt handling **from first principles**, without relying on vendor frameworks, and verify your work using **QEMU and GDB**.

---

2. Learning Outcomes

On successful completion of this assignment, you will be able to:

1. Explain the Cortex-M reset and boot process
2. Construct a valid vector table and `Reset_Handler`
3. Perform runtime initialization (`.data` copy, `.bss` zero)
4. Write and reason about a custom linker script
5. Build and run bare-metal firmware without an RTOS
6. Use QEMU and GDB to debug early boot code
7. Implement and verify a working interrupt (SysTick)

---

3. Fixed Platform and Constraints

3.1 Platform

- **MCU family:** STM32F4 (Cortex-M4F)
- **QEMU machine:** `olimex-stm32-h405`
- **Host OS:** Windows 10 or Linux (Ubuntu x86_64)

3.2 Restrictions

You **must not** use:

- STM32Cube HAL or LL libraries
- CubeMX-generated code
- Vendor startup files or linker scripts
- Any RTOS

You **may** use:

- GNU ARM Embedded Toolchain
- CMSIS *core* headers (optional, register definitions only)

---

4. Assignment Tasks

Part A — Vector Table and Reset Handler

Create a startup file that:

1. Defines a vector table at the start of Flash
2. Sets the initial stack pointer
3. Implements `Reset_Handler`
4. Provides default handlers for exceptions

You must verify using **GDB** that:

- SP is initialized from the vector table
- PC jumps to `Reset_Handler`
- The reset vector address has the Thumb bit set

Part B — Runtime Initialization

Inside `Reset_Handler`, implement:

1. Copy `.data` from Flash (load address) to SRAM (run address)
2. Zero-initialize `.bss`
3. Call `main()`

In `main()`, verify correctness using:

```
int initialized = 123;
int uninitialized;
```

You must demonstrate that:

- `initialized == 123`
- `uninitialized == 0`

Part C — Custom Linker Script

Write a **custom linker script** defining:

- FLASH and RAM regions
- Sections:
  - `.isr_vector`
  - `.text`
  - `.rodata`
  - `.data`
  - `.bss`

You must define the following symbols:

```
_estack
_sidata
_sdata
_edata
_sbss
_ebss
```

You must submit and explain a **linker map file**, describing:

- Vector table placement
- `.data` load vs run addresses
- Flash and SRAM usage

Part D — Observable Output on QEMU

Since GPIO is not available in the QEMU STM32 model, you must implement **one** of the following:

- Semihosting output (recommended), or
- USART serial output

The output must occur **after runtime initialization** and prove that `main()` has been reached.

Part E — SysTick Interrupt

Configure and enable **SysTick**:

1. Set reload value
2. Enable SysTick interrupt
3. Implement `SysTick_Handler`

The handler must:

- Update a `volatile` counter
- Periodically produce observable output or be verifiable via GDB

---

5. Deliverables

Submit a single repository or ZIP containing:

```
/src
  main.c
  semihosting.h
/startup
  startup_stm32f4.s
/ld
  linker.ld
Makefile
README.md
firmware.map
```

ALL CODE MUST BE COMMENTED AT THE RATE 30 LINES OF COMMENTS PER 100 LINES OF SOURCE CODE

README must include:

- Memory map used
- QEMU run and debug commands
- Boot sequence explanation (6–10 bullets)
- GDB evidence (screenshots or logs)

---

6. Evaluation (100 Marks)

Component	Marks
Build reproducibility	10
Vector table & reset	15
Runtime initialization	20
Linker script	15
Observable output	10
SysTick interrupt	20
Debug evidence	10

Appendix A — Installing and Using QEMU

A1. Windows 10

Install QEMU

Download from: https://www.qemu.org/download/
Select **Windows x64 installer** and enable **Add to PATH**.

Verify:

```
qemu-system-arm --version
```

Install ARM Toolchain

Download from:
https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads

Verify:

```
arm-none-eabi-gcc --version
```

Run

```
qemu-system-arm -M olimex-stm32-h405 -kernel firmware.bin -nographic
```

A2. Linux (Ubuntu)

```
sudo apt update
sudo apt install qemu-system-arm gcc-arm-none-eabi gdb-multiarch make
```

Verify:

```
qemu-system-arm --version
```

A3. Debugging with GDB

```
qemu-system-arm -M olimex-stm32-h405 -kernel firmware.bin \
  -S -gdb tcp::3333 -nographic
```

In another terminal:

```
arm-none-eabi-gdb firmware.elf
(gdb) target remote :3333
(gdb) monitor reset halt
```

Appendix B — Build System and Makefile

B1. Why Make?

Bare-metal firmware requires explicit control over:

- CPU/ABI flags
- Linker scripts
- Section placement

Using `make` exposes these mechanisms clearly.

B2. Build Stages

1. **Compile:** `.c`, `.s` → `.o`
2. **Link:** `.o` → `.elf` using linker script

3. **Extract:** `.elf` → `.bin` using `objcopy`

B3. Reference Makefile

```
CC      = arm-none-eabi-gcc
OBJCOPY = arm-none-eabi-objcopy
SIZE    = arm-none-eabi-size

TARGET = firmware

C_SRCS = src/main.c
S_SRCS = startup/startup_stm32f4.s
LD_SCRIPT = ld/linker.ld

OBJS = $(C_SRCS:.c=.o) $(S_SRCS:.s=.o)

CFLAGS = -mcpu=cortex-m4 -mthumb -O0 -g \
         -Wall -ffreestanding -nostdlib \
         -mfpu=fpv4-sp-d16 -mfloat-abi=hard

ASFLAGS = $(CFLAGS)
LDFLAGS = -T $(LD_SCRIPT) -nostdlib -Wl,-Map=$(TARGET).map

all: $(TARGET).bin

%.o: %.c
        $(CC) $(CFLAGS) -c $< -o $@

%.o: %.s
        $(CC) $(ASFLAGS) -c $< -o $@

$(TARGET).elf: $(OBJS)
        $(CC) $(OBJS) $(LDFLAGS) -o $@
        $(SIZE) $@

$(TARGET).bin: $(TARGET).elf
        $(OBJCOPY) -O binary $< $@

clean:
        rm -f $(OBJS) $(TARGET).elf $(TARGET).bin $(TARGET).map
```

Appendix C — Minimal Semihosting Output

```
#define SEMIHOSTING_SYS_WRITE0 0x04

static inline int semihosting_call(int reason, void *arg)
{
    int value;
    __asm volatile (
        "mov r0, %1\n"
```

```
        "mov r1, %2\n"
        "bkpt 0xAB\n"
        "mov %0, r0\n"
        : "=r"(value)
        : "r"(reason), "r"(arg)
        : "r0", "r1", "memory"
    );
    return value;
}

static inline void sh_puts(const char *s)
{
    semihosting_call(SEMIHOSTING_SYS_WRITE0, (void*)s);
}
```

Run QEMU with:

```
-semihosting-config enable=on,target=native
```

Appendix D — Reset-to-main() Execution Timeline

```
Reset
 |
 |--> SP = *(FLASH + 0x00)
 |--> PC = *(FLASH + 0x04)
 |
Reset_Handler
 |
 |--> copy .data
 |--> zero .bss
 |
main()
 |
SysTick enabled
 |
SysTick_Handler()
```

Appendix E — Skeleton Source Code (Hint)

startup/startup_stm32f4.s

```
.syntax unified
.cpu cortex-m4
.thumb

.extern _estack
.extern _sidata
```

```
.extern _sdata
.extern _edata
.extern _sbss
.extern _ebss

.global g_pfnVectors
.global Reset_Handler

.section .isr_vector,"a",%progbits
g_pfnVectors:
  .word _estack
  .word Reset_Handler

.section .text.Reset_Handler,"ax",%progbits
Reset_Handler:
  /* copy .data */
  ldr r0, =_sidata
  ldr r1, =_sdata
  ldr r2, =_edata
1: cmp r1, r2
   bcc 2f
   b 3f
2: ldr r3, [r0], #4
   str r3, [r1], #4
   b 1b
3:
  /* zero .bss */
  ldr r1, =_sbss
  ldr r2, =_ebss
  movs r3, #0
4: cmp r1, r2
   bcc 5f
   b 6f
5: str r3, [r1], #4
   b 4b
6:
  bl main
7: b 7b
```

---

src/main.c

```
#include "semihosting.h"
#include <stdint.h>

int initialized = 123;
int uninitialized;

int main(void)
{
    sh_puts("Boot OK\r\n");

    if (initialized == 123 && uninitialized == 0)
        sh_puts("Data/BSS verified\r\n");

    while (1);
```

}